

Introduction

Programming for Economists

Jeppe Druedahl and Brigitte Hochmuth

Autumn 2026

1. Intended learning goals
2. Course plan
3. Exam
4. Active learning
5. Programming in practice
6. Summing up

Intended learning goals

- **In a nutshell:**

1. Summarize and visualize economic data
 - gain flexibility and automation compared to Excel
 - build upon Descriptive Economics
 - work with larger and messier data sets
2. Numerically solve and simulate economic models
 - use fewer simplifying assumptions than on paper
 - connect programming to Micro and Macro
 - move from model-user to *model-builder*

Intended learning goals

- **In a nutshell:**

1. Summarize and visualize economic data

- gain flexibility and automation compared to Excel
- build upon Descriptive Economics
- work with larger and messier data sets

2. Numerically solve and simulate economic models

- use fewer simplifying assumptions than on paper
- connect programming to Micro and Macro
- move from model-user to *model-builder*

- **Tool:** *Python*, a general-purpose programming language (open source)

1. Web development (not covered)

2. Data science and machine learning (covered as tools)

3. Scientific computing and research (main focus)

Why programming matters

- Economic data rarely arrive as clean Excel files
 - you often need to download, clean, merge and document data
- Programming makes work reproducible
 - the same code can update a figure when new data arrive
- Models become easier to explore
 - change a parameter, re-solve the model, plot the result
- These skills matter for later courses, theses, internships and policy work

Why learn this when AI can help?

- AI can help with syntax, debugging and first drafts of code
- AI does not know whether your data definition or model assumption is right
- You must be able to read, test and defend the code you submit
- In some settings, for example confidential data servers, AI tools may not be allowed
- The goal is not to memorize syntax. The goal is to understand the workflow.

Course plan

- **Lectures:**

- 1: Introduction
- 2-4: Python basics, functions, classes and NumPy
- 5: Printing and plotting
- 6-8: Optimization, root-finding, interpolation and simulation
- 9-10: Descriptive Economics and data project
- 11-12: Solow model, Walras economy and calibration
- 13: Recap and exam preparation

- **Lectures:**

- 1: Introduction
- 2-4: Python basics, functions, classes and NumPy
- 5: Printing and plotting
- 6-8: Optimization, root-finding, interpolation and simulation
- 9-10: Descriptive Economics and data project
- 11-12: Solow model, Walras economy and calibration
- 13: Recap and exam preparation

- **Classes:**

- Week 37: installation, notebooks, Problem Set 0 and DataCamp
- Weeks 38-41: DataCamp plus small accompanying problem sets
- Weeks 43-51: problem sets, projects and exam preparation

September 9

- There is no in-person lecture on Wednesday, September 9
- Use the week for structured self-study
 1. make sure Python, Anaconda and VSCode work
 2. work on Problem Set 0
 3. start the assigned DataCamp chapters
- Lecture 2 on September 16 begins with a short recap and coding exercises

- **Web-site:** ProgEcon course website
 1. installation guides and VSCode guide
 2. calendar and practical information
- **Course material in two GitHub repositories**
 1. ProgEcon-lectures: slides, notebooks, course plan and projects
 2. ProgEcon-exercises: problem sets and solutions
- **Need-to-know:** click on Code and Download ZIP

Exam

- **Requirements:**
 1. complete the assigned DataCamp tasks
 2. Project 1: data project
 3. Project 2: model project
- **Exam:** portfolio of revised projects plus take-home assignment
- **Grading:** pass or fail
- **Groups:** projects and exam can be done individually or in groups of up to 4
 - groups may consist of students from different exercise classes, or “holds”

DataCamp and early problem sets

- Access DataCamp here: [DataCamp course link](#)
- You must complete the following DataCamp courses
 1. Introduction to Python
 2. Intermediate Python
- Optional extra practice
 1. Introduction to Functions in Python
 2. Introduction to NumPy
 3. Intermediate Object-Oriented Programming in Python
- Early exercise classes combine DataCamp with small accompanying problem sets

DataCamp deadlines

All deadlines are Sundays at 23:59.

Deadline	Required progress
----------	-------------------

Sep 13	Introduction to Python
--------	-------------------------------

	Ch. 1: <i>Python Basics</i>
--	-----------------------------

	Ch. 2: <i>Python Lists</i>
--	----------------------------

Sep 20	Introduction to Python
--------	-------------------------------

	Ch. 3: <i>Functions and Packages</i>
--	--------------------------------------

	Ch. 4: <i>NumPy</i> Course completed
--	---

Sep 27	Intermediate Python
--------	----------------------------

	Ch. 1: <i>Matplotlib</i>
--	--------------------------

	Ch. 2: <i>Dictionaries & Pandas</i>
--	---

Oct 4	Intermediate Python
-------	----------------------------

	Ch. 3: <i>Logic, Control Flow and Filtering</i>
--	---

	Ch. 4: <i>Loops</i>
--	---------------------

Oct 11	Intermediate Python
--------	----------------------------

	Ch. 5: <i>Case Study: Hacker Statistics</i> Course completed
--	---

- **Objective:** use Python to answer a simple economic question with data
- **Content:**
 1. import data from an online source
 2. clean and structure the data
 3. visualize the data and explain what the figures show
 4. apply methods from Descriptive Economics
- **Structure:** self-contained notebook and project files
- **Deadline:** Sunday, November 29, 23:59

Model project

- **Objective:** use Python to solve and analyze a simple economic model
- **Content:**
 1. describe the algorithm
 2. solve or simulate the model
 3. visualize results across parameter choices
 4. analyze one extension of the baseline model
- **Structure:** self-contained notebook and project files
- **Deadline:** Sunday, December 13, 23:59

Active learning

- To learn programming you need to work on actual problems yourself
- We give you an overview and a set of tools
- DataCamp gives practice with syntax and simple examples
- Problem sets force you to apply the tools to economics
- You have to experiment on your own
- You have to acquire new tools on your own in later courses and at work

- Copilot and LLMs can be useful assistants
 - translate words into code
 - explain error messages
 - suggest examples
- Use them after thinking about the problem yourself
- Verify that you understand the answer before you use it
- For graded work, you are responsible for the final submission

- Numerical analysis is a *complement*, not a substitute for math
- Three central steps:
 1. mathematical problem → construct algorithm
 2. algorithm → write code
 3. code → present results
- The set of potential errors is very large
 1. clear structure reduces bugs
 2. testing helps find bugs
 3. documentation helps remove bugs
- Programming is also structuring, testing, documenting and collaborating

- **Lectures:**

1. attend lectures and participate actively
2. try out the code yourself

- **Exercises:** short introduction, then problem sets, DataCamp or project work

1. solve small tasks
2. fill in missing code
3. find errors
4. ask questions

- **Between lectures and classes:**

1. go through lecture notebooks
2. solve the problem set
3. experiment with your own ideas

Getting help

- Programming often means removing many small errors, one at a time
- Everybody forgets syntax. Trial-and-error and testing are part of the work.
- Many sources can help
 1. documentation
 2. course material
 3. Copilot and LLMs
 4. fellow students
 5. teaching assistants and teachers
- Help each other and keep feedback constructive

Programming in practice

Python distribution and VSCode

- We work with Python 3.x
- Suggested environment:
 1. distribution: Anaconda
 2. code editor: VSCode
 3. notebooks: Jupyter notebooks inside VSCode
- Today we keep the examples simple
 1. open the course repository
 2. run a notebook cell
 3. change one line of code
 4. check whether the result changes as expected

Examples in this lecture

- **Small data example:** calculate and plot a simple economic time series
- **Small simulation example:** generate random numbers and summarize the result
- **Small model example:** change a parameter and see how the model outcome changes
- The point is not to understand every detail today
- The point is to see what you will be able to do by the end of the course

Further material

- The old course material is available as supplementary material
- It is useful if you want more practice or more advanced examples
- It is not the main path through this course
- The notebooks and exercise classes define what we focus on this semester

Summing up

Summing up

- You should now have an idea of:
 1. why programming and numerical analysis matter for economists
 2. what you will learn in this course
 3. why active participation is required
 4. how DataCamp, classes, projects and the exam fit together

Your to-do list

1. First priority: log in to DataCamp.com
2. Second priority: install Anaconda and VSCode
3. Third priority: download lecture and exercise material
4. Fourth priority: open the introductory notebook and run the example code yourself
5. Fifth priority: complete the first DataCamp chapters by Sunday, September 13